

PROGRAMA INTEGRAL DE FORMACIÓN EN PYTHON

CURSO 1: FUNDAMENTOS BÁSICOS DE PYTHON (NIVEL 1
(PRINCIPIANTES))

Clase 06: Diccionarios y Mapeos Clave-Valor

«La Agenda Telefónica y el Expediente Médico»

Estructuras asociativas de alto rendimiento: diccionarios (dict), asociación clave-valor, métodos
(.get(), .keys(), .items()), anidamiento y similitud con JSON.

Instructor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Nivel del Curso: Principiante Absoluto | **Python:** 3.10+ | **Licencia:** MIT

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre **wisrovi SUITE** en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Estructura Pedagógica de Cada Guía

Cada documento de esta serie está diseñado rigurosamente bajo el estándar de ingeniería de software profesional:

- **Fundamento Conceptual y Metáfora Intuitiva:** Anclaje visual del concepto abstracto a la realidad.
- **Diagrama de Flujo y Arquitectura Mental:** Representación visual de cómo fluye la información.
- **Código de Nivel Profesional:** Sintaxis limpia, comentarios línea a línea y tipado estático (PEP 484).
- **Patrones y Gotchas:** Errores comunes que cometen los desarrolladores y cómo evitarlos.

Índice General de la Sesión

Sección	Contenido y Enfoque Temático	Página
Capítulo 1	Fundamentos y Metáfora Conceptual: Mapeos Asociativos y la Estructura Clave-Valor	Pág. 4
Capítulo 2	Diagrama de Flujo y Arquitectura: Arquitectura de un Diccionario y Búsqueda por Hash	Pág. 5
Capítulo 3	Implementación Práctica en Python: Sistema de Gestión de Inventario con Diccionarios	Pág. 6
Capítulo 4	Patrones Avanzados, Gotchas y Debugging: Gotchas Clásicos con Diccionarios	Pág. 7
Cierre	Conclusiones, Notas del Mentor y Agradecimiento	Pág. 8
Anexos	Bibliografía Oficial y Enlaces de Profundización	Pág. 9

Objetivos de Aprendizaje (Competencias Clave)



Competencia Conceptual

Comprender la indexación por clave semántica en lugar de posición numérica y la eficiencia $O(1)$ de las tablas hash.



Competencia Práctica

Construir modelos de datos complejos con diccionarios anidados y procesar registros estructurados.

Requisitos Previos y Entorno Recomendado

Para aprovechar al máximo esta sesión se recomienda tener instalado Python 3.10 o superior, Visual Studio Code con la extensión oficial de Python configurada, y haber completado las lecturas y ejercicios de las sesiones anteriores de la ruta formativa.

1. Mapeos Asociativos y la Estructura Clave-Valor

Buscar un dato por su posición (índice 4) es poco intuitivo; en el mundo real buscamos por nombre, correo o ID.

☀ **Metáfora Central: La Agenda Telefónica y el Expediente Médico**

Un diccionario es como tu agenda del teléfono: no buscas a tu mamá por el número de orden en que la agregaste, buscas la etiqueta 'Mamá' (la clave) y obtienes su número de teléfono (el valor).

Principios Teóricos y Modelo Mental

Las claves en un diccionario deben ser únicas e inmutables (comúnmente strings o ints). Los valores pueden ser de cualquier tipo, incluidas listas u otros diccionarios.

La búsqueda en un diccionario es instantánea (tiempo constante $O(1)$) gracias al algoritmo interno de tabla hash.

⚡ **Regla de Oro en Python**

Nunca accedas a una clave con `dict['clave']` si no estás 100% seguro de que existe; usa `dict.get('clave', valor_por_defecto)` para evitar `KeyError`.

2. Arquitectura de un Diccionario y Búsqueda por Hash

Cómo Python mapea claves alfanuméricas a ubicaciones de memoria específicas.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Python aplica una función hash a la clave (ej: hash('email')).	Clave -> Hash ID numérico
2. Evaluación	Localiza el casillero exacto en la tabla hash de memoria.	Búsqueda O(1)
3. Transformación	Recupera o modifica el valor asociado sin recorrer toda la estructura.	Lectura/Escritura inmediata
4. Retorno / Salida	Permite serialización directa hacia y desde formato JSON para APIs web.	Compatibilidad universal

Visualización Mental

Los diccionarios son el equivalente en Python a los objetos de JavaScript o los registros de bases de datos NoSQL.

3. Sistema de Gestión de Inventario con Diccionarios

Manipulación de registros de productos con métodos `.get()`, `.items()` y anidamiento:

main.py (Python 3.10+)

UTF-8 • PEP 8 Compliant

```
inventario = {
    "PROD-001": {"nombre": "Teclado Mecánico", "precio": 85.0, "stock": 12},
    "PROD-002": {"nombre": "Mouse Ergonómico", "precio": 45.0, "stock": 0}
}

# Acceso seguro con .get()
sku_buscado = "PROD-001"
producto = inventario.get(sku_buscado, None)

if producto:
    print(f"Producto: {producto['nombre']} | Stock: {producto['stock']} uds")

# Iteración completa de claves y valores
for sku, datos in inventario.items():
    disponible = "En Stock" if datos["stock"] > 0 else "Agotado"
    print(f"[{sku}] {datos['nombre']} -> {disponible}")
```

Análisis del Código Fuente

Se utiliza una estructura anidada dict-of-dicts, acceso resiliente con `get()` y desempaqueo de tuplas con el método `.items()`.

4. Gotchas Clásicos con Diccionarios

Errores habituales al consultar y mutar diccionarios:

⚠ Gotcha Frecuente (Trampa de Principiante)

Consultar una clave inexistente con corchetes (`dict['inexistente']`) provoca un `KeyError` que detiene el programa.

Patrón Recomendado vs Antipatrón

✗ Antipatrón / Mal Código

```
user = {'nombre': 'Leo'}  
print(user['edad']) # KeyError: 'edad'
```

✓ Patrón Pythonic / Correcto

```
user = {'nombre': 'Leo'}  
print(user.get('edad', 0)) # Retorna 0 de forma segura
```

🛡 Consejo de Resiliencia en Producción

Utiliza dictionary comprehensions (`{k: v for k, v in ...}`) para filtrar y transformar diccionarios en una sola línea.

5. Resumen Ejecutivo y Conclusiones

Comprendes a fondo los diccionarios, la estructura clave-valor y el modelado de entidades del mundo real.



Logro Alcanzado en esta Clase

Capacidad para manipular datos estructurados complejos, payloads JSON y configuraciones de software.

Notas del Instructor y Sigüientes Pasos

En la próxima clase estudiaremos las Funciones (def): el arte de empaquetar código reutilizable y modular.



Mensaje de Agradecimiento

Muchas gracias por tu entusiasmo, disciplina y dedicación al participar en este programa formativo. La programación es un superpoder que transforma vidas cuando se ejerce con constancia y curiosidad. ¡Nos vemos en la próxima sesión para seguir construyendo juntos!

«El código más elegante es aquel que cualquiera puede entender y nadie necesita reescribir.»

6. Fuentes Bibliográficas y Recursos de Estudio

Para profundizar y consolidar los conocimientos adquiridos en esta clase, consulta las siguientes referencias:

Recurso / Fuente	Descripción Temática	Enlace Oficial
Documentación Oficial de Python	Referencia canónica del lenguaje y librería estándar	docs.python.org/3/
PEP 8 - Style Guide for Python	Guía oficial de estilo, formato e indentación	peps.python.org
Real Python Tutorials	Artículos técnicos y patrones de desarrollo moderno	realpython.com
Python Type Checking (PEP 484)	Anotaciones de tipo y análisis estático en Python	docs.python.org/typing
Suite Open Source wisrovi	Paquetes Python para orquestación y alto rendimiento	github.com/wisrovi

Canales de Consulta y Comunidad

Si encuentras dificultades al replicar el código o tienes dudas sobre la arquitectura de los ejercicios, no dudes en abrir un Issue en el repositorio oficial del curso en GitHub o participar activamente en nuestras sesiones grupales de mentoría.



Desafío de Autoestudio Recomendado

Crea una función que reciba una lista de palabras y devuelva un diccionario con la frecuencia de aparición de cada palabra.